mTLS · TLS · 그리고 Envoy가 암호를 풀었다 다시 감을 때

평문 HTTP → TLS → mTLS → 프록시 종료 — 순서와 그 대가

Cilium 스터디 보충 자료

오늘 답할 네 가지 질문

통신 순서

1. 평문 HTTP는 어떤 순서로 오가나?
2. TLS가 끼면 그 순서가 어떻게 바뀌나?
3. mTLS는 거기서 무엇이 더 붙나?

그리고 대가

4. Envoy가 TLS를 **복호화했다 재암호화**하면

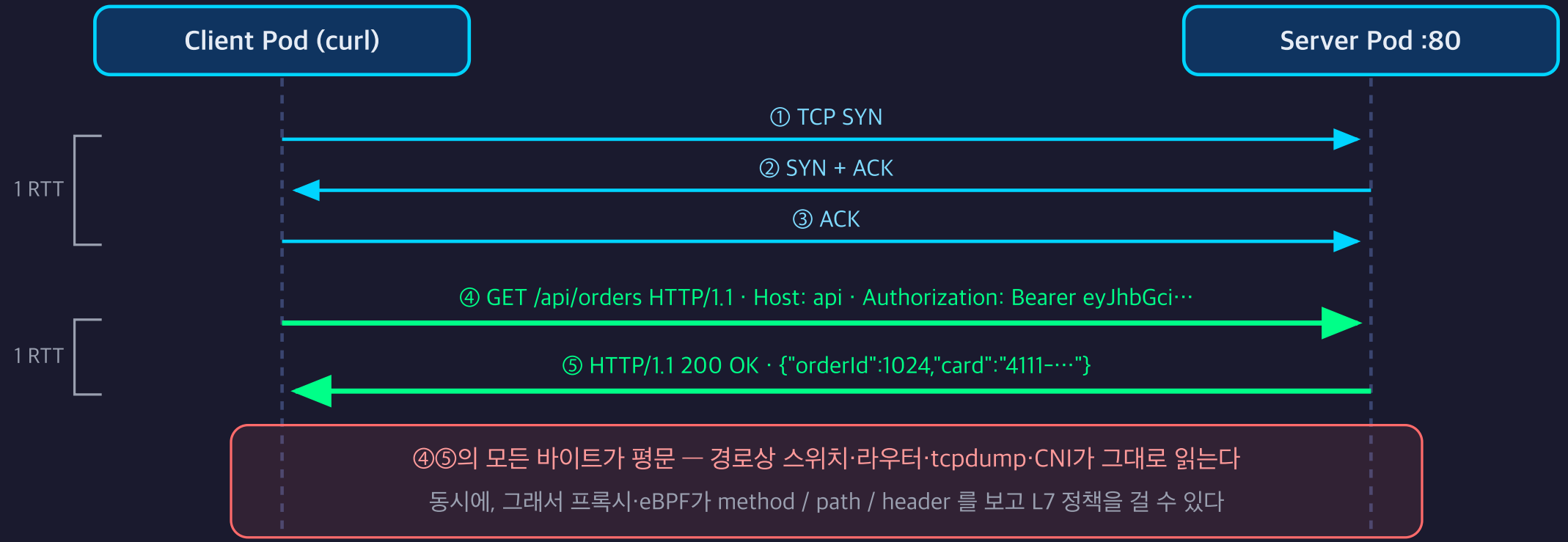
- 무엇이 평문으로 노출되고
- 무엇이 느려지고
- 무엇이 망가지나?

> 한 줄 요약: **L7 가시성과 종단간 암호화는 동시에 성립할 수 없다.** 선택지는 "누가, 얼마나 오래, 얼마나 멀리 평문을 들고 있는가"뿐이다.

01 — 평문 HTTP

아무것도 감추지 않을 때의 순서

평문 HTTP — 통신 순서



첫 요청 바이트를 보내기까지 **1 RTT** · 응답까지 **2 RTT**

평문의 양면성 — 위험이자 전제

❌ 위험

- Authorization 토큰·쿠키·PII가 그대로
- 경로상 어느 홉에서든 **수정** 가능 (무결성 없음)
- 상대가 진짜 그 서버인지 **확인 불가**

✅ 전제

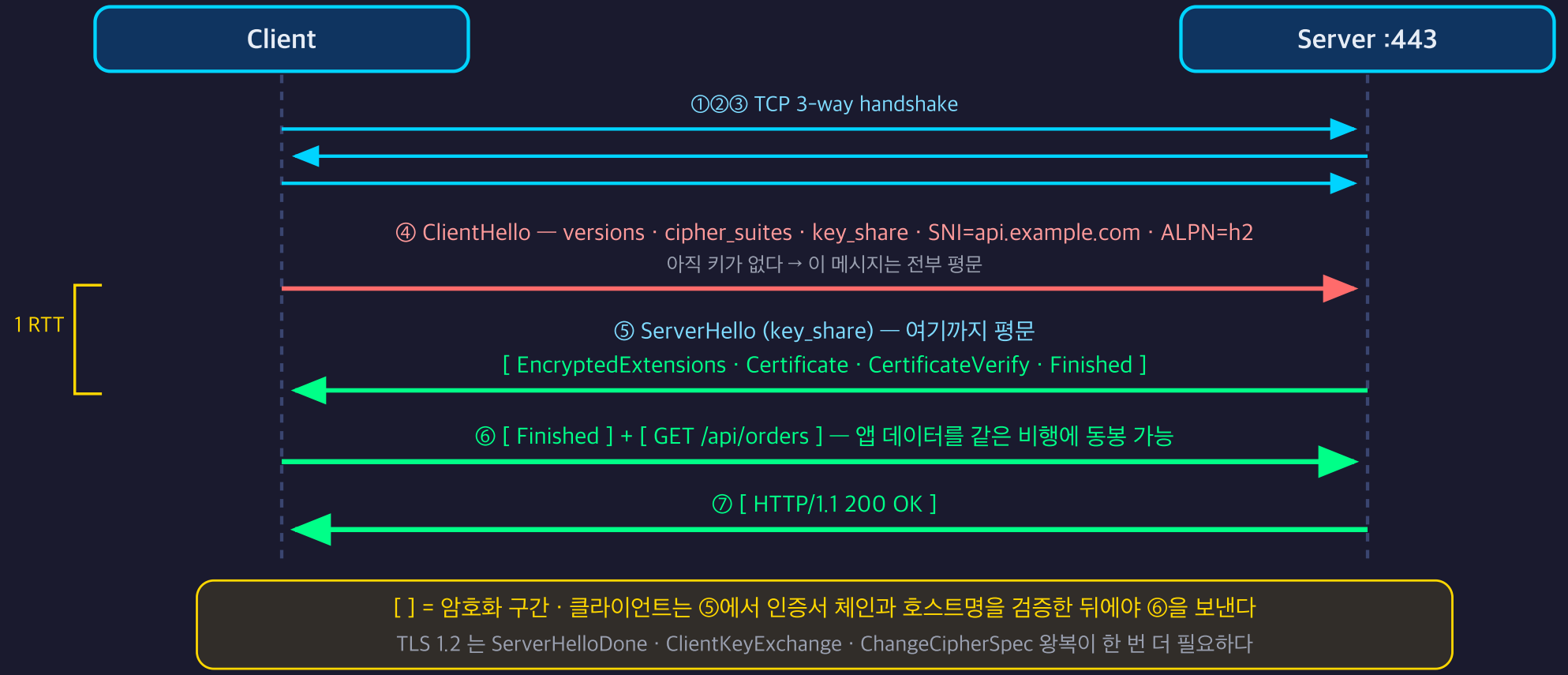
- Envoy가 `method` / `path` / `header` 파싱 가능
- L7 AuthorizationPolicy, 경로 라우팅, 재시도
- 요청 단위 메트릭 · 분산 트레이싱

> 보안이 요구하는 것과 관측·정책이 요구하는 것이 **정확히 반대**다. TLS는 위쪽을 해결하면서 아래쪽을 파괴한다.

02 — TLS

순서가 어떻게 바뀌고, 무엇이 남는가

TLS 1.3 — 통신 순서


Client
Server :443
①②③ TCP 3-way handshake
④ ClientHello — versions · cipher_suites · key_share · SNI=api.example.com · ALPN=h2
아직 키가 없다 → 이 메시지는 전부 평문
1 RTT
⑤ ServerHello (key_share) — 여기까지 평문
[EncryptedExtensions · Certificate · CertificateVerify · Finished]
⑥ [Finished] + [GET /api/orders] — 앱 데이터를 같은 비행에 동봉 가능
⑦ [HTTP/1.1 200 OK]
[ ] = 암호화 구간 · 클라이언트는 ⑤에서 인증서 체인과 호스트명을 검증한 뒤에야 ⑥을 보낸다
TLS 1.2 는 ServerHelloDone · ClientKeyExchange · ChangeCipherSpec 왕복이 한 번 더 필요하다

RTT 비교 — 무엇이 얼마나 늘어나나

구간	평문 HTTP	HTTPS (TLS 1.2)	HTTPS (TLS 1.3)	TLS 1.3 재개(0-RTT)
TCP 핸드셰이크	1 RTT	1 RTT	1 RTT	1 RTT
TLS 핸드셰이크	—	2 RTT	1 RTT	0 RTT
첫 요청 바이트까지	**1 RTT**	**3 RTT**	**2 RTT**	**1 RTT**
바이트당 대칭 암복호	없음	양단 각 1회	양단 각 1회	양단 각 1회
연결당 비대칭 연산	없음	서명 검증 1회	서명 검증 1회	생략 가능

- **0-RTT 재개**는 replay 공격에 취약하므로 멱등 요청(GET)에만 써야 한다.
- RTT 비용은 **연결당 1회**다. keep-alive / HTTP2 멀티플렉싱으로 상각된다.
- 진짜 지속 비용은 RTT가 아니라 **바이트당 대칭 암복호**와, 잠시 뒤 볼 **프록시 홉**이다.

암호화해도 남는 것 vs 사라지는 것

핸드셰이크 첫 패킷 (ClientHello)



이후 애플리케이션 데이터 패킷



남는 것: IP · 포트 · SNI · ALPN · 패킷 크기 / 타이밍 ⟶ L3/L4 정책은 여전히 가능, L7 정책은 불가능

그래서 L7 정책은 원천적으로 불가능하다

```
CiliumNetworkPolicy — L4까지
  toPorts: [{ port: "443", protocol: TCP }]        ✅ 커널 eBPF에서 판단 가능

CiliumNetworkPolicy — L7
  rules:
    http:
      - method: "GET"
        path: "/api/orders"                        ❌ 암호문 안이라 볼 수 없음
```

- eBPF는 패킷 하나를 훅에서 즉시 판단한다. **복호화 키도, 스트림 재조합 상태도 없다.**
- 정책이 `path`를 보려면 **누군가 반드시 복호화해야 한다.** 예외 없다.
- 그 "누군가"가 프록시 — Istio에선 사이드카 Envoy, Cilium에선 노드당 `cilium-envoy`.

> L7 정책을 켜는 순간 당신은 **의도적으로 중간자(MITM)를 세우는 것**이다. 문제는 그 대가가 뭐냐다.

03 — mTLS

한쪽이 아니라 양쪽을 증명한다

TLS vs mTLS — 누가 누구를 증명하나

TLS — 단방향 인증



서버 인증서 제출

클라이언트가 신뢰 CA로 검증

클라이언트 신원은? → 네트워크 계층엔 없음

앱 계층 JWT · 비밀번호 · API key 에 위임

mTLS — 양방향 인증



서버 인증서 ⇄ 클라이언트 인증서

양쪽 모두 개인키 소유를 서명으로 증명

네트워크 계층에서 양쪽 신원이 확정된다

인가 기준 = 위조 가능한 IP → 암호학적 워크로드 신원

mTLS 핸드셰이크 — 무엇이 더 붙나



노란 줄 세 개가 mTLS의 전부 — **RTT는 늘지 않는다.** 늘어나는 건 비대칭 연산 2회와 flight 크기.

메시가 mTLS를 원하는 진짜 이유

신원 = 인증서, IP 아님

```
SAN: URI, spiffe://cluster.local
        /ns/shop/sa/frontend
```

- Pod IP는 **재사용·위조·수시 변경**
- SPIFFE ID는 ServiceAccount에 고정

운영 특성

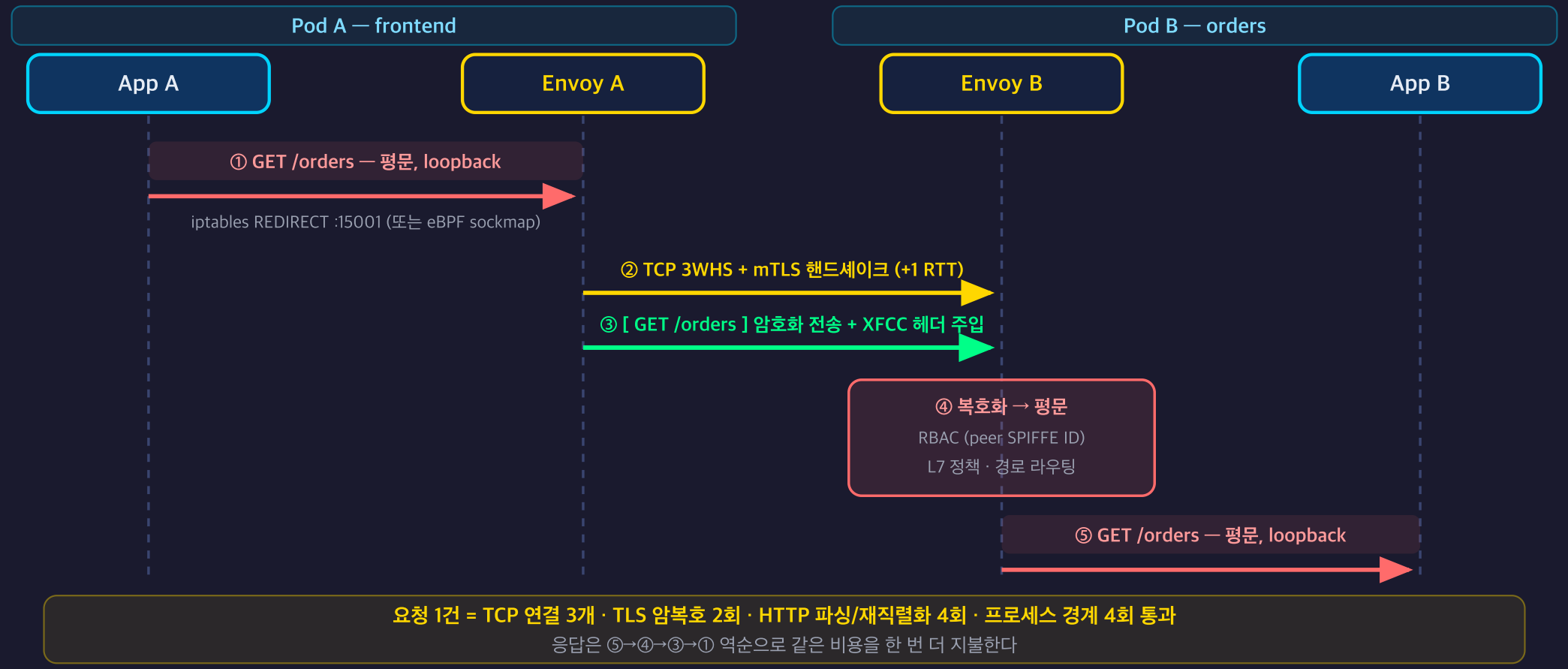
- **단명 인증서** (24h 내외) + 자동 rotation
- **SDS**로 UDS를 통해 메모리 전달 — 디스크에 키를 쓰지 않음
- 앱 코드 **무수정**

> 얻는 것: L3/L4 정책이 "어느 IP에서 왔나"가 아니라 **"어느 워크로드가 왔나"**로 바뀐다. 그런데 이걸 얻으려고 세운 Envoy가, 바로 다음 섹션의 모든 문제의 원인이 된다.

04 — Envoy가 풀었다 다시 감을 때

TLS termination & re-origination 의 청구서

사이드카 전체 경로 — 평문 구간이 어디인가



문제 ① — E2E가 아니라 hop-by-hop이다

평문이 존재하는 곳

- Pod A의 loopback (App A ↔ Envoy A)
- **Envoy A 프로세스 메모리**
- **Envoy B 프로세스 메모리**
- Pod B의 loopback (Envoy B ↔ App B)

공격 표면

- Pod 내 `CAP_NET_RAW` → `tcpdump -i lo`
- `kubectl debug` 임시 컨테이너, ephemeral sidecar
- Envoy 침해 시 **평문 + 신원 위조** 동시 획득
- 코어 덤프 · 메모리 스크래핑

"서비스 메시를 켰으니 종단간 암호화된다"는 **틀린 문장**이다. 정확히는 Envoy-to-Envoy 구간만 암호화된다. 신뢰 경계가 앱이 아니라 **Pod 샌드박스**로 이동했을 뿐이다.

문제 ② — 비용

항목	프록시 없음	사이드카 mTLS
요청 경로의 TCP 연결 수	1	**3**
바이트당 암복호	0	**2회** (A 암호화, B 복호화)
HTTP 파싱 / 재직렬화	0	**4회**
첫 요청까지 RTT	1	**3** (TCP + mTLS 1.3)
Pod당 상주 메모리	0	Envoy 1개 분량 추가
컨트롤플레인 설정 전파	없음	Pod 수에 비례해 xDS 증가

- 지연은 대개 **밀리초 단위**로 잡히지만, 문제가 되는 건 평균이 아니라 **p99 꼬리**다.
- 사이드카가 **Pod 수만큼** 존재한다 → 노드당 수용 Pod 감소, 기동 지연, 롤링 업데이트 폭증.

> 실측 수치는 워크로드·CPU·커넥션 재사용률에 따라 크게 달라진다. **구조적으로 늘어나는 항목**을 기억하는 게 숫자를 외우는 것보다 유용하다.

문제 ③ — 의미 손실과 운영 함정

연결 의미가 바뀐다

- 앱이 보는 src IP = `127.0.0.1`
- 원 신원은 `X-Forwarded-Client-Cert` 헤더로 재주입 → **헤더를 신뢰해야 함**
- 앱이 XFCC를 직접 받지 않도록 막아야 스푸핑 방지

프로토콜 충실도

- h2 → h1.1 다운그레이드 변환, 헤더 케이스 정규화
- trailer · WebSocket · 양방향 gRPC 스트리밍 엣지케이스
- 파싱 못 하는 프로토콜은 raw TCP 패스스루로 강등 → **L7 기능 상실**

수명주기 경합 — Envoy보다 앱이 먼저 뜨면 `connection refused`, 앱보다 Envoy가 먼저 죽으면 트래픽 블랙홀. (k8s 1.29 native sidecar로 완화)

인증서 배관 — SDS, cert rotation, CA rotation, 시계 오차 → 핸드셰이크 실패. 디버깅이 앱 로그가 아니라 프록시 로그로 이동한다.

문제 ④ — 근본 딜레마

앱이 진짜 E2E TLS를 하면

- 평문은 App A · App B 안에만 존재 ✅
- Envoy는 **TCP 바이트 스트림**만 봄
- 경로 라우팅 · L7 RBAC · HTTP 메트릭 **전부 불가**
- 얻는 건 L4 통계와 연결 수뿐

메시가 TLS를 종료하면

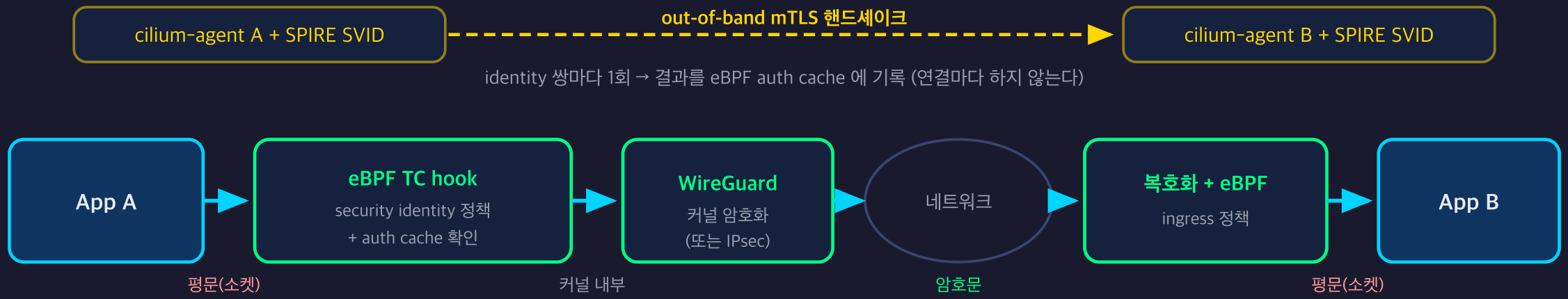
- L7 기능 **전부 가능** ✅
- 대신 평문이 **프록시 메모리**에 존재
- 신뢰 경계가 앱 → Pod 샌드박스로 확대
- 프록시가 곧 최우선 공격 표적

둘 다는 원리적으로 불가능하다. L7 판단은 평문 없이는 정의되지 않기 때문이다.
실제 설계 선택지는 딱 세 가지: 평문을 누가 쥐는가 · 얼마나 오래 쥐는가 · 얼마나 멀리 옮기는가.

05 — Cilium은 어떻게 비껴가나

복호화하지 않고도 되는 부분은 복호화하지 않는다

Cilium 경로 — 프록시 홉이 없다



userspace 프록시 0개 · 추가 TCP 연결 0개 · 복호화된 평문이 userspace 에 존재하지 않는다

단, 이 경로로는 L3/L4 정책만 가능하다 — HTTP method / path 는 여전히 볼 수 없다

그래도 L7 + HTTPS는 결국 명시적 MITM

```yaml
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
spec:
  egress:
  - toPorts:
    - ports: [{ port: "443", protocol: TCP }]
      terminatingTLS:                # Pod → Envoy 구간 TLS 종료 (Pod가 내부 CA를 신뢰해야 함)
        secret: { namespace: default, name: tls-orig }
      originatingTLS:                # Envoy → 목적지 구간 TLS 재개시
        secret: { namespace: default, name: tls-dest }
      rules:
        http:
        - method: "GET"
          path: "/v1/orders"         # 이 판단은 평문에서만 가능
```

- Cilium도 예외가 아니다. `terminatingTLS` + `originatingTLS` = **문서화된 TLS interception**.
- 차이는 **범위**: Pod마다가 아니라 **노드당 `cilium-envoy` 1개**, 그리고 **L7 룰이 걸린 흐름만** 우회.

세 모델 비교

	Istio 사이드카	Istio Ambient	Cilium
L4 암호화 주체	Pod마다 Envoy	노드당 ztunnel (Rust)	커널 WireGuard / IPsec
데이터 경로 프록시 홉	2	2 (L4) / 4 (Waypoint)	0 (L3/L4)
신원	SPIFFE 인증서	SPIFFE 인증서	security identity + SPIFFE(auth, beta)
mTLS 핸드셰이크 시점	연결마다	연결마다	identity 쌍당 1회, out-of-band
userspace 평문	항상	L4는 없음	L7 룰 있을 때만
L7 정책	항상 가능	Waypoint 배치 시	노드당 Envoy로 우회 시
성숙도	안정	안정	L3/L4 안정, mutual auth beta

축은 하나다: **평문을 만드는 지점을 몇 개 두고, 얼마나 좁게 유지하는가.**

한 장 요약

평문 HTTP = 1 RTT · 전부 노출 · 그래서 L7 정책 가능

TLS 1.3 = +1 RTT · IP·포트·SNI만 남고 L7은 사라짐

mTLS = RTT 그대로 · 서명 2회 추가 · 신원이 IP → 인증서

Envoy 종료 = 연결 3개 · 암복호 2회 · 파싱 4회 · **평문 지점 2곳**

> L7 가시성과 종단간 암호화는 **동시에 성립하지 않는다.**
>
> 좋은 설계 = 평문을 **누가 · 얼마나 오래 · 얼마나 멀리** 들고 있는지를 최소화하는 것.